

RS ASSIGNMENT#01

19/08/2025 8:55 AM

QUESTION #01

NABHA KHAN 23K-0914

Figure A – Content-Based Recommender System

1. This diagram illustrates a content-based filtering pipeline where recommendations are driven by the similarity of item features.
2. The User Profile is created by analyzing a user's previous likes, ratings, or consumed items.
3. Simultaneously, the Item Analyzer extracts metadata and key features from items (e.g., genre, keywords, attributes).
4. Both user profiles and item profiles are stored in permanent storage, serving as a long-term memory for the system.
5. The Recommendation Engine sits at the core, matching items against the stored user profile.
6. The system computes relevancy scores based on how closely an item's features align with the user's preferences.
7. Items with the highest similarity are shortlisted and ranked for recommendation.
8. The output is a personalized recommendation list presented to the user.
9. The user's feedback, whether explicit (ratings) or implicit (clicks), is looped back into the user profile for refinement.
10. The uniqueness of this figure is its closed feedback loop between user preferences, item features, and storage, showing how the system keeps learning over time.

Figure B – Architecture of Docear's Research Paper RS

1. This diagram shows a domain-specific RS focused on academic literature, unlike generic RS architectures.
2. Users interact by uploading mind-maps or searching for PDFs directly via the Docear web service.
3. A Spider Tool continuously crawls the web to gather academic PDFs for the system.
4. Each PDF undergoes pre-processing: extracting text, title, authors, abstract, and references, and assigning a unique Docear ID.
5. These processed PDFs are indexed and stored in a large corpus that powers the system's recommendations.
6. From a user's uploaded mind-map, the system generates a Mind-Map Model representing research interests.
7. This is transformed into a User Model that captures the topics and directions relevant to that researcher.
8. Two strategies drive recommendations: General (common, trending papers) and Personalized (content-based filtering).
9. To balance relevance and diversity, Docear selects the top 50 matches and randomly recommends 10.
10. The uniqueness here lies in its integration of mind-maps, literature crawling, and hybrid recommendation tuned specifically for researchers.

Figure C – Vector-Based Approach

1. This diagram depicts how RS can operate using vector representations of users and items.
2. The User Profile Builder collects explicit (ratings, favorites) and implicit (clicks, purchases) data.
3. If detailed item features are missing, the entire item itself can be used as an interest vector.
4. The system organizes interactions like buys, ratings, and evaluations into a graph-like intermediate structure.
5. The Item Analyzer extracts metadata such as category, brand, or tags from items.
6. These details are mapped into user-feature relationships that connect users with item attributes.
7. In the Recommendation Engine, these mappings are converted into numerical vectors.
8. User profiles and item vectors are compared using similarity measures like cosine similarity.
9. Items with the highest similarity scores are recommended to the user.
10. The uniqueness of this figure is its vectorization process, where both users and items are embedded into the same mathematical space for direct comparison.

Figure D – Graph-Based Approach

1. This diagram demonstrates a graph-structured RS, where everything is modeled as nodes and edges.
2. The graph contains three main nodes: users, items, and features.
3. Edges connect nodes, such as a user linked to a movie they watched or a movie linked to its genre.
4. The graph evolves into a network of relationships, forming a rich representation of interactions.
5. Recommendation works by starting from a user node and traversing connected paths in the graph.
6. The system explores direct edges (e.g., movies watched) and indirect edges (e.g., movies with similar features).
7. The distance in the graph reflects how strong a recommendation is—the closer, the stronger.
8. Graph algorithms such as random walks or community detection can be applied to find relevant items.
9. The final recommendation list is composed of items that are well-connected or proximal to the user in the graph.
10. The uniqueness of this figure is its explicit visualization of relationships between users, items, and features, enabling richer contextual recommendations than vectors alone.

Figure E – RS in the Fashion Industry

1. This diagram highlights a domain-adapted RS tailored for fashion-specific personalization.
2. Inputs include user attributes such as body shape, lifestyle, occasion, and even scenery preferences.
3. Past behavior like purchases, ratings, and browsing history enrich personalization.
4. The system incorporates style understanding, covering garment, outfit, item, and accessory design.
5. It emphasizes aesthetic aspects like color, texture, and pattern matching.
6. It also integrates fashion trend prediction, keeping recommendations updated with current styles.
7. Algorithms rely heavily on AI/ML, both supervised (classification) and unsupervised (clustering).
8. Automation extends to clothes, accessories, makeup, and hairstyles.
9. Supporting technologies include computer vision (image recognition), NLP (text), and voice-based interfaces.
10. The uniqueness of this figure is its multi-modal integration of AI and fashion design principles, addressing not just preference matching but also style and trend forecasting.

QUESTION #02

a) $MAE = \frac{|1-2-4| + |2-2-3| + |3-4-3| + |0-1-1| + |2-5-3| + |3-2-2| + |0-5-2| + |2-4| + |4-5-4| + |0-1-2| + |0-2-2| + |0-5-3|}{4 \times 3}$

= 1.396

$$RMSE = \sqrt{\frac{(1-2-4)^2 + (2-2-3)^2 + (3-4-3)^2 + (0-1-1)^2 + (2-5-3)^2 + (3-2-2)^2 + (0-5-2)^2 + (2-4)^2 + (4-5-4)^2 + (0-1-2)^2 + (0-2-2)^2 + (0-5-3)^2}{4 \times 3}}$$

= 1.597

b) $MAE = \frac{|2-3-3| + |0-4| + |3-2-4| + |4-5-4| + |1-2-4| + |0-3| + |2-2-3| + |3-4-3| + |0-1-1| + |0-6-2| + |2-5-3| + |3-2-2| + |0-5-2| + |0-3-3| + |2-4| + |4-5-4| + |0-1-2| + |1-9-3| + |0-2-2| + |0-5-3|}{5 \times 4}$

= 1.548

$$RMSE = \sqrt{\frac{(2-3-3)^2 + (0-4)^2 + (3-2-4)^2 + (4-5-4)^2 + (1-2-4)^2 + (0-3)^2 + (2-2-3)^2 + (3-4-3)^2 + (0-1-1)^2 + (0-6-2)^2 + (2-5-3)^2 + (3-2-2)^2 + (0-5-2)^2 + (0-3-3)^2 + (2-4)^2 + (4-5-4)^2 + (0-1-2)^2 + (1-9-3)^2 + (0-2-2)^2 + (0-5-3)^2}{5 \times 4}}$$

= 1.836

QUESTION #03

TRAVELER A = [Hotel1, Hotel2, Hotel3]

TRAVELER B = [Route1, Route2, Route3, Route4]

TRAVELER C = [Meal1, Meal2, Meal3, Meal4]

[Hotel4, Hotel5, Hotel6, Hotel7]

[Route5, Route6, Route7, Route8]

[Meal5, Meal6, Meal7, Meal8]

A

ORDER	MEDIC	IS RATED	PRES@i	calc in AP@k	RECALL@k
1	H4	no	$\frac{0}{3} = 0$		$\frac{0}{3} = 0$
2	H2	yes	$\frac{1}{3} = 0.33$	✓	$\frac{1}{3} = 0.33$
3	H5	no	$\frac{1}{3} = 0.33$		$\frac{1}{3} = 0.33$
4	H2	yes	$\frac{2}{3} = 0.67$	✓	$\frac{2}{3} = 0.67$
5	H6	no	$\frac{2}{3} = 0.67$		$\frac{2}{3} = 0.67$
6	H7	no	$\frac{2}{3} = 0.33$		$\frac{2}{3} = 0.67$

$PRES@i = \frac{\text{num of relevant items till } i}{i}$

$RECALL@i = \frac{\text{num of relevant items till } i}{\text{total relevant items}}$

$MAP@k = \frac{\text{sum of AVG precision across all users}}{\text{total users}}$

$AP@k = \frac{1}{r \text{ or min}(|R_u|, k)} \sum_{i=1}^k (PRES@i \cdot rel(i)) = \frac{1}{3} \times (0.33 + 0.5) = 0.333$

$MAR@k = \frac{\text{sum of recall across all users}}{\text{total users}}$

$AR@k = \frac{1}{r} \sum_{i=1}^k (RECALL@i \cdot rel(i)) = \frac{1}{3} \times (0.33 + 0.67) = 0.333$

B

ORDER	MEDIC	IS RATED	PRES@i	calc in AP@k	RECALL@k
1	R2	yes	$\frac{1}{1} = 1$	✓	$\frac{1}{1} = 0.25$
2	R5	no	$\frac{1}{2} = 0.5$		$\frac{1}{2} = 0.25$
3	R3	yes	$\frac{2}{3} = 0.67$	✓	$\frac{2}{3} = 0.5$
4	R3	yes	$\frac{2}{4} = 0.75$	✓	$\frac{2}{4} = 0.75$
5	R6	no	$\frac{2}{5} = 0.6$		$\frac{2}{5} = 0.75$

$AP@k = \frac{1}{4} (1 + 0.67 + 0.75) = 0.604$

$AR@k = \frac{1}{4} (0.25 + 0.5 + 0.75) = 0.375$

C

ORDER	MEDIC	IS RATED	PRES@i	calc in AP@k	RECALL@k
1	M2	yes	$\frac{1}{1} = 1$	✓	$\frac{1}{1} = 0.25$
2	M5	no	$\frac{1}{2} = 0.5$		$\frac{1}{2} = 0.25$
3	M3	yes	$\frac{2}{3} = 0.67$	✓	$\frac{2}{3} = 0.5$
4	M3	yes	$\frac{2}{4} = 0.75$	✓	$\frac{2}{4} = 0.75$
5	M6	no	$\frac{2}{5} = 0.6$		$\frac{2}{5} = 0.75$

$AP@k = \frac{1}{4} (1 + 0.67 + 0.75) = 0.604$

$AR@k = \frac{1}{4} (0.25 + 0.5 + 0.75) = 0.375$

$MAP@k = \frac{0.333 + 0.604 + 0.604}{3} = 0.514$

$MAR@k = \frac{0.333 + 0.375 + 0.375}{3} = 0.361$

QUESTION#04

```
def cosine_similarity(person a, person b, all_ratings):
    ratings_a = all_ratings.get(person a, {})
    ratings_b = all_ratings.get(person b, {})

    common = set(ratings_a.keys()) & set(ratings_b.keys())
    if not common:
        return 0

    dot = sum(ratings_a[item] * ratings_b[item] for item in common)
    norm_a = sum(ratings_a[item] ** 2 for item in common) ** 0.5
    norm_b = sum(ratings_b[item] ** 2 for item in common) ** 0.5

    if norm_a == 0 or norm_b == 0:
        return 0
    return dot / (norm_a * norm_b)

def neighborhood_recommendation(top_n, neighbor_count, target_user, all_items,
all_ratings):
    user_data = all_ratings.get(target_user, {})
    not_rated = [item for item in all_items if item not in user_data]

    scores = {}
    for item in not_rated:
        users_who_rated = [u for u in all_ratings if item in all_ratings[u]
and u != target_user]

        if not users_who_rated:
            scores[item] = 0
            continue

        user_similarities = []
        for other in users_who_rated:
            sim = cosine_similarity(target_user, other, all_ratings)
            user_similarities.append((sim, all_ratings[other][item]))

        user_similarities.sort(reverse=True)
        top_neighbors = user_similarities[:neighbor_count]

        if top_neighbors:
            weighted_sum = sum(sim * rating for sim, rating in top_neighbors)
            sim_sum = sum(sim for sim, _ in top_neighbors)
            scores[item] = weighted_sum / sim_sum if sim_sum != 0 else 0
        else:
            scores[item] = 0

    sorted_items = sorted(scores.items(), key=lambda x: x[1], reverse=True)
    return [item for item, score in sorted_items[:top_n]]

# example usage
if __name__ == "__main__":
    all_ratings = {
        'U1': {'I1': 4.0, 'I2': 3.0},
        'U2': {'I1': 5.0, 'I3': 4.0},
        'U3': {'I2': 4.5, 'I4': 5.0, 'I5': 4.0}
    }

    all_items = ['I1', 'I2', 'I3', 'I4', 'I5']
    target_user = 'U1'
    top_n = 3
    neighbor_count = 2

    result = neighborhood_recommendation(top_n, neighbor_count, target_user,
all_items, all_ratings)
    print(f"Top {top_n} recommended items for {target_user}: {result}")
```

Output

Top 3 recommended items for U1: ['I4', 'I3', 'I5']

PS C:\Users\Bira\Documents>